Operations

Run it day to day, upgrade it, fix problems.

For: Administrators and engineers

One chapter of the handbook. The full handbook, with every chapter, is handbook.pdf in the same folder.

Day-to-day running, upgrades, backups and troubleshooting.

Everyday commands

```
docker compose ps                                  # what's running and healthy
docker compose logs -f --tail=100 policy-api       # follow one service
docker compose restart policy-api                  # after editing .env
docker compose stop && docker compose up -d        # full restart
scripts/check_containment.sh                       # after any change
```

Optional services need their profile: `docker compose --profile escalate up -d escalate`.

Health

Check	Healthy looks like
`curl -s http://127.0.0.1:8100/health`	`{"ok":true,"demo_mode":false}` (if `demo_mode` is true, real models are **not** in use)
`docker compose ps`	`db`, `policy-api` and `granite-gateway` show `healthy`
`curl -s $GRANITE_URL/v1/models`	Lists `granite-4.2-30b`. Granite runs on its own server, so check it there (and with whoever runs it), not with `nvidia-smi` here
Recent runs	`SELECT status, count(*) FROM runs WHERE created_at > now() - interval '1 day' GROUP BY 1;` shows few `failed`
Job queue	`curl -s -H "Authorization: Bearer $POLICY_API_KEY" http://policy-api:8000/queue` from inside the network: `oldest_wait_s` near 0 and `dead` not rising

The job queue

Every check is a job in the `jobs` table. `policy-api` runs a worker that takes jobs one at a time per free slot (`WORKER_CONCURRENCY`, default 2) and renews a lease on each while it works (`JOB_LEASE_SECONDS`, default 60).

- **Restarts and crashes don't lose checks.** If `policy-api` restarts, it takes back its own jobs at once. If it dies or the server reboots, the lease runs out and the job is picked up again. The check **carries on after the last finished step**, using its saved checkpoint; only the step that was running is repeated. People waiting in Open WebUI see "Starting again after an interruption…". See Resume after a crash.

- **Temporary problems are retried** (a model server restarting, the database briefly away), up to `JOB_MAX_ATTEMPTS` (default 3) with growing waits. After that the job is `dead` and the person sees "failed several times in a row".
- **Problems with the question itself are not retried** (no such policy, nothing changed).
- **More capacity:** add workers with `docker compose run -d --name worker-2 policy-api python -m app.worker`, or raise `WORKER_CONCURRENCY`. Workers never take the same job twice. Set `WORKER_ENABLED=0` on `policy-api` if you want only separate workers to run checks.

A check that is waiting for a person (`REVIEW_GATE=1`) is **not** a running job. It has finished its job and sits in `runs` as `needs_review`. See Find a paused check.

Dead jobs keep their last error:

```
SELECT id, kind, payload, attempts, last_error, finished_at FROM jobs WHERE status = 'dead' ORDER
BY id DESC LIMIT 20;
-- Retry one after fixing the cause:
UPDATE jobs SET status = 'pending', attempts = 0, run_at = now() WHERE id = <id>;
UPDATE runs SET status = 'queued', error = NULL, finished_at = NULL WHERE id = '<payload
run_id>';
-- The retry carries on from the last saved step. To start the check again from the beginning
instead,
-- delete its checkpoints first (see "Start a check again from scratch" below).
-- Housekeeping (monthly):
DELETE FROM jobs WHERE status = 'done' AND finished_at < now() - interval '30 days';
```

Find a paused check

With `REVIEW_GATE=1`, a check that has "maybe" sections stops and waits. In the chat it shows as a list with a check number. To see every paused check:

```
SELECT left(id::text, 8) AS check_number, head_policy, subset, requested_by, created_at,
       jsonb_array_length(jsonb_path_query_array(result, '$.rows[*] ? (@.status == "Maybe")')) AS
waiting
FROM runs WHERE status = 'needs_review' ORDER BY created_at;
```

Or ask for one check, from inside the network (the short number works on this route):

```
curl -s -H "Authorization: Bearer $POLICY_API_KEY" http://policy-api:8000/runs/3f9a1c2e
# "status": "needs_review", and "review": the sections still waiting
```

A paused check uses no worker and no model, and it waits for ever. Nothing times it out. To see one in the chat, ask the assistant for the result of that check number; it shows the list again, and the person types `decide <check number> ...`. To answer from the command line instead (for example for someone who left), send the same request the filter sends:

```
curl -s -X POST -H "Authorization: Bearer $POLICY_API_KEY" -H "Content-Type: application/json" \
  -d '{"decisions":[{"policy_id":"SAF-150","clause":"3.1","affected":true}],"reviewed_by":"Your
Name"}' \
  http://policy-api:8000/runs/3f9a1c2e/review
```

It answers 202. Use your own name in `reviewed_by`: it is stored in the result and shown as "Confirmed by". Sections you leave out stay "maybe". If you would rather drop the pause for a check nobody will decide, there is no button for it. Leaving it is harmless: it uses nothing.

Resume after a crash

Usually you do nothing. After a crash, a reboot or `docker compose restart policy-api`:

1. A restarted worker takes its own jobs back at once. After a hard crash, the lease (60 seconds) runs out and any worker takes the job.
2. The graph reads its checkpoint and continues from the first step that did not finish.
3. The person waiting sees "Starting again after an interruption…", then the answer.

To check, look at the queue and at the run:

```
SELECT id, kind, status, attempts, run_at, last_error FROM jobs WHERE status IN ('pending',
'running') ORDER BY id;
SELECT status, progress FROM runs WHERE id = '<run id>';
SELECT checkpoint_id, metadata->>'step' AS step FROM checkpoints WHERE thread_id = '<run id>'
ORDER BY checkpoint_id;
```

The last query lists the saved steps for the check. The thread id is the run id. If a check stays stuck after `policy-api` is healthy again, the job is probably `dead`; use the retry SQL under the job queue.

Start a check again from scratch

If a check's saved state is bad (for example after restoring an older database backup, or the same error repeats after a fix), delete its checkpoints, then queue it again:

```
DELETE FROM checkpoint_writes WHERE thread_id = '<run id>';
DELETE FROM checkpoint_blobs  WHERE thread_id = '<run id>';
DELETE FROM checkpoints       WHERE thread_id = '<run id>';
```

Then ask the question again in the chat. That makes a new check with a new number. The checkpoint tables are made by `policy-api` on start; do not create or change them by hand.

How long a run takes

Roughly: number of changed sections × about 20–40 related clauses × time per verdict ÷ `JUDGE_CONCURRENCY`. For example, 10 changed sections with 30 clauses each is 300 verdicts; at 16 at a time and about 3 s each, that's about one minute. Raise `JUDGE_CONCURRENCY` if the Granite server isn't busy; lower it if verdicts time out.

Backups

Back up the database daily. The models and the policy source files are recoverable from their original sources; runs and verdicts are not.

```
docker compose exec -T db pg_dump -U policy -Fc policy > backups/policy-$(date +%F).dump
# Restore into an empty database:
docker compose exec -T db pg_restore -U policy -d policy --clean --if-exists < backups/policy-
YYYY-MM-DD.dump
```

The checkpoint tables are in the same database, so this backs them up too. Restoring an older dump brings back older checkpoints; a check that finished after the dump has no checkpoint, and one that was paused may need to be asked again.

Test a restore every quarter on a spare machine. Also back up the `openwebui` volume (users and the tool configuration), and keep `.env` in your secrets vault.

Upgrades

Change one thing at a time, and run the evaluation after anything that touches the models.

1. Read the release notes for the component.
2. Bring the new image or weights in as in setup.md; record and check hashes.
3. Change the tag in `.env` (or the folder in `compose.yaml`).
4. `docker compose up -d <service>`
5. `scripts/check_containment.sh`
6. Smoke test (testing.md), and evaluate if a model changed.
7. Roll back by restoring the old tag and running `docker compose up -d <service>`.

Granite upgrades happen on the Granite server, not here. After any change there (vLLM version, flags, model files), run `curl -s $GRANITE_URL/v1/models`, then the evaluation. To change the key or address, edit `GRANITE_API_KEY` / `GRANITE_URL` in `.env` and run `docker compose up -d granite-gateway`.

Open WebUI upgrades can change the tool API. Afterwards, attach a file and ask a question. If the tool says no file was attached, check how the new release passes `__files__` and adjust `_attached_file` in `openwebui/policy_impact_tool.py`.

policy-api upgrades: rebuild outside the environment with a new `APP_VERSION`, carry the image in, and set `APP_VERSION` in `.env`. Checks in progress during the restart are picked up again automatically. Database changes in `policy_api/app/migrations/` are applied on start, once each.

Changing the embedding model

The `clauses.embedding` column has a fixed size (768). To switch to a model with a different size:

```
docker compose exec db psql -U policy -c "DROP INDEX clauses_embedding_idx;
  ALTER TABLE clauses ALTER COLUMN embedding TYPE vector(<new size>) USING NULL;
  CREATE INDEX clauses_embedding_idx ON clauses USING hnsw (embedding vector_cosine_ops);"
# Put the new model in models/embedding, restart embed, then re-ingest every policy:
docker compose up -d embed && docker compose run --rm policy-api python -m app.ingest /policies
```

Update `vector(768)` in `db/init.sql` too, so new installs match. Re-run the evaluation.

Moving a model to different GPUs

Run `python3 scripts/setup_gpus.py` again and give the new GPU numbers. Listing several GPUs for `escalate` (or another vLLM model run by this stack) spreads that model across them (tensor parallelism); the script sets `ESCALATE_TP` and the other size settings to match. Then `docker compose up -d escalate` (or whichever service moved). Granite's GPUs are managed on its own server.

Troubleshooting

Symptom	Likely cause	Fix
policy-api exits with `Missing settings`	`.env` incomplete	Fill in the named variables
policy-api exits with `at least 32 characters`	Short key	Regenerate with `openssl rand -hex 32`
`400 Invalid host header`	The Host name isn't in `ALLOWED_HOSTS`	Add the name the proxy sends (`policy-api`) or your hostname
Chat says "The policy service could not be reached"	policy-api down, or the tool's `API_URL` / `API_KEY` Valve is wrong	`docker compose ps` ; check the Valves
Chat says "checking model is not responding"	The Granite server is down or overloaded, or `granite-gateway` cannot reach it	`docker compose logs granite-gateway` ; run `curl -s $GRANITE_URL/v1/models` ; lower `JUDGE_CONCURRENCY`
Many "could not be read" verdicts	Structured output not working with this vLLM version	Check the Granite server's vLLM version supports `response_format` JSON schema; check its logs
The assistant answers without calling the tool	Function calling not set to Native, or the tool isn't enabled on the model	setup.md step 4
"No numbered sections were found" for a Word file	Word automatic numbering	Save with typed numbers, or as PDF (how-to.md)
Download link gives 403	Link expired (72 h) or copied incompletely	Ask the assistant again
Download link gives a connection error	`PUBLIC_BASE_URL` isn't reachable from users' browsers	Set it to the proxy's public address and restart policy-api
A run stays "running"	It's large, or Granite is slow	Check `progress` on the run; the tool stops waiting after 30 minutes but the run continues
A check says it needs a decision, and nobody answers	`REVIEW_GATE=1` and nobody typed `decide …`	Find a paused check
Typing `decide …` does nothing, or the assistant shows a hint	The upload filter is off or not Global, the check number is wrong, or a section is not on the list	setup.md step 3; copy the number from the assistant's message; use only the listed sections
"This check is not waiting for a decision"	Someone else already answered it, or it has finished	Ask the assistant for the result of that check number
Ingest skips files	No ID, no numbered sections, or unreadable type	The skip reason is printed for each file

Symptom	Likely cause	Fix
`check_containment.sh` fails	A service can reach the internet	security.md